Dt.:
Pg.:

Memory Management

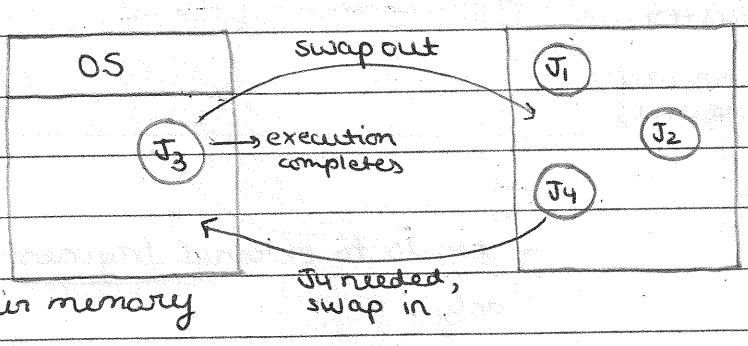
Main memory refers to physical memory that is internal memory of computer. (also known as RAM). The computer is able to charge only data that is in mainmemory.

All programs are loaded in main memory for execution. Sometimes complete program is loaded into main memory, but sometimes a certain part or routine of program is loaded in main memory, only when it is called a program, this mechanism is called dynamic loading.

Also, at times one program is dependent on some other program. In such a case, rather than loading all the dependent programs, CPU links the dependent programs to main executing program when it is required. This mechanism is called dynamic linking.

Swapping

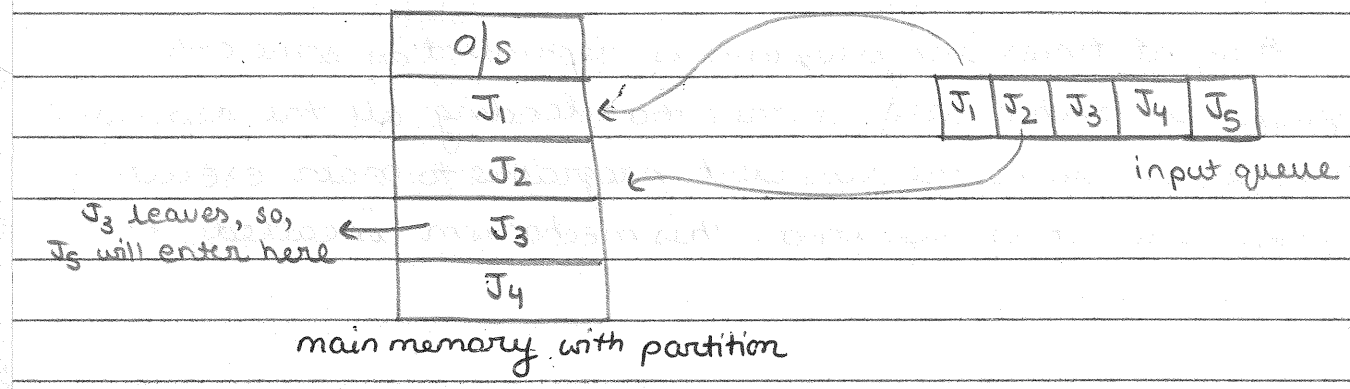
A process needs to be in memory for execution. But sometimes there is not enough main memory to hold all currently active processes in a timesharing system. So excess process are kept on disk and brought in to run dynamically. swapping is a process of bringing in each process in main memory, running it for a while and then putting it back to disk.



Dt.:
Pg.:

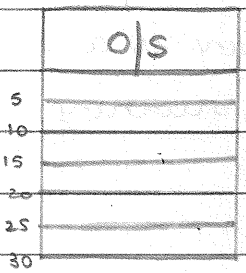
Contiguous Memory Allocation

In contiguous memory allocation each process is contained in single contiguous block of memory. memory is divided into several fixed size partitions. Each partition contains exactly one process. When a partition is free, a process is selected from input queue and loaded in it. The free blocks of memory are called holes. The set of holes is searched to determine which hole is best to allocate.



MFT (multiprogramming with fixed no. of tasks)

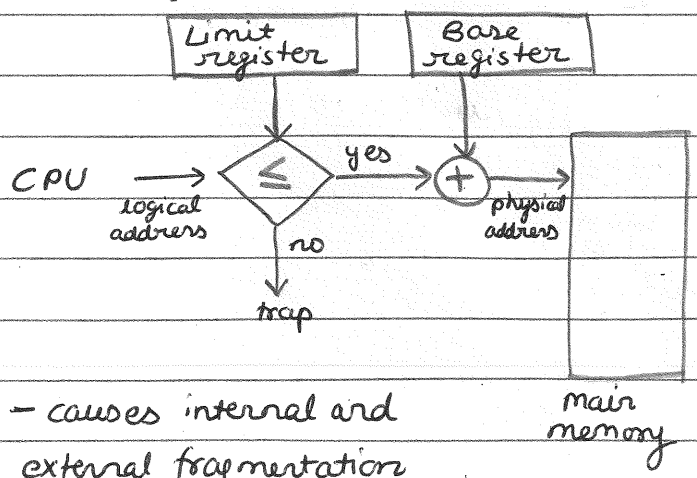
- We have predefined partition



degree of multiprogramming = 6 = no. of division

Now we should have two bound register

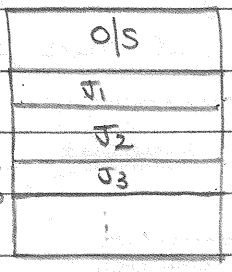
Lower bound (base register) — Upper bound (limit register)



- causes internal and external fragmentation both.

MVT (multiprogramming with variable no. of task)

- we donot have predefined partition we create partition according size of program.



- leads to external fragmentation only.
- to solve this we can do compaction. but it is costly because CPU is free at that time. so, we need to look for another method.

Fragmentation.

- occurs in dynamic memory allocation system when most of the free blocks are too small to satisfy the request. It is generally termed as inability to use available memory.

External Fragmentation - when free holes exists to satisfy a request but is non-contiguous ie. memory is fragmented into large no. of small holes.

Internal Fragmentation - At times physical memory is broken to fixed size blocks and memory is allocated in unit of blocks sizes. The memory allocated may be slightly larger than requested memory. The difference b/w allocated and required memory is known as internal fragmentation.

Memory Allocation.

Memory allocation is a process by which computer programs are assigned memory of space. It is of three types:

First Fit: The first hole that is big enough is allocated to program.

Best Fit: The smallest hole that is suits best is allocated to program

Worst Fit: The largest hole that is big enough is allocated to program.

Paging

A solution to fragmentation is Paging. Paging is a memory management mechanism that allows the physical address space of a process to be non-contagious. Here physical memory is divided into blocks of equal size of called ~~Pages~~ Frames.

The pages belonging to certain process are loaded into available memory frames.

Every process is divided into no. of Pages

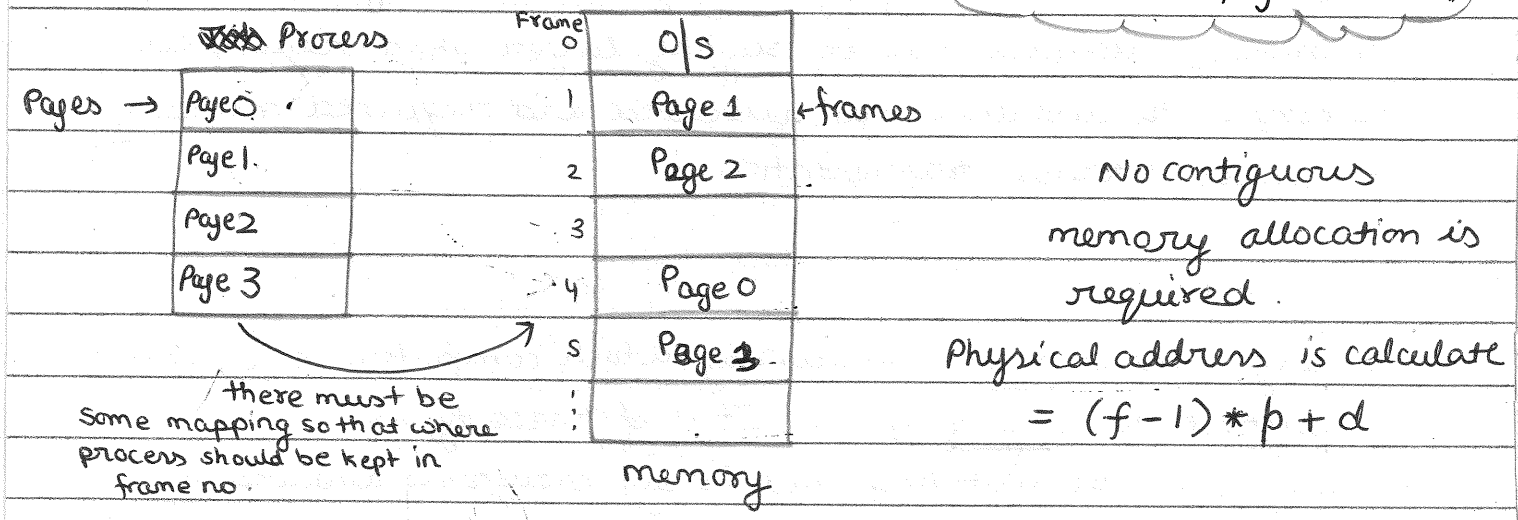
Pages are usually implemented as a set of associated registers.

Paging → fixed size.

Page table is a <u>data structure</u> used by virtual memory system in a computer operating system to store the mapping b/w virtual address and physical addresses.
CPU produces logical address (also known as virtual address) while Physical address that actually exists on memory.

Process → divided into → Pages
Memory → divided into → Frames
page size = frame size

Yes, internal fragmentation
No external fragmentation.

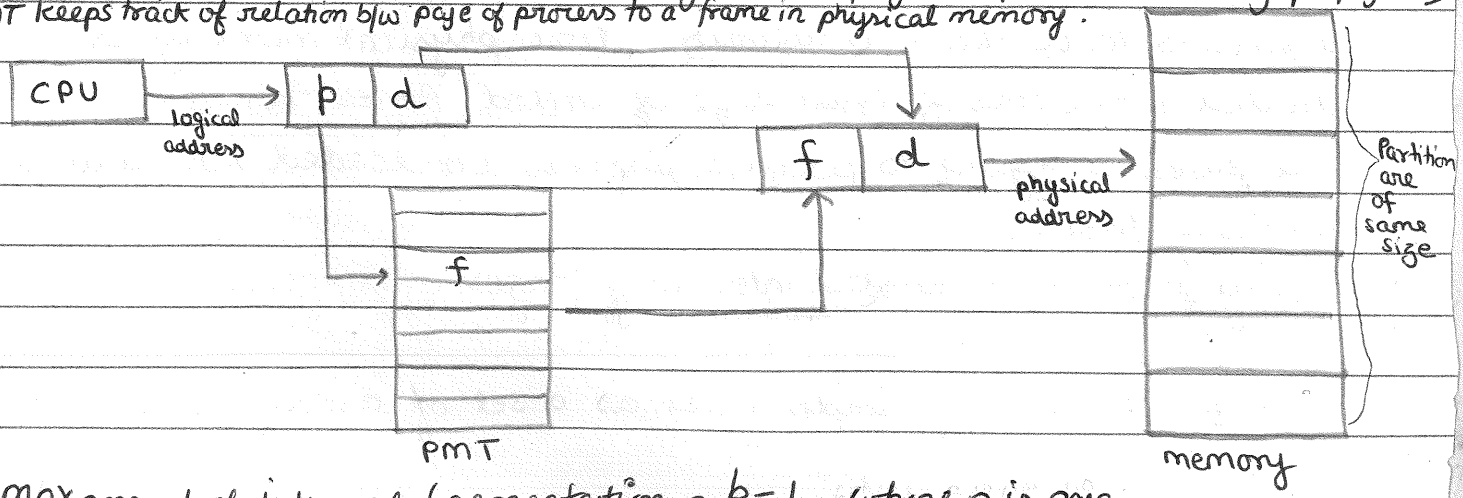


No contiguous memory allocation is required.
Physical address is calculate
$= (f-1) * p + d$

P → page size. L → logical address
Logical address (L) is generated by CPU. From this we need to find p, d. (page no and offset)

$p = L \text{ div } P$. and $d = L \bmod P$

frame no is calculated by PMT (page map table) using p (page no)
PMT keeps track of relation b/w page of process to a frame in physical memory.



max amount of internal fragmentation = $p-1$ where p is page no.

segment → ~~variable~~ different size.

Dt.: Pg.: Delta

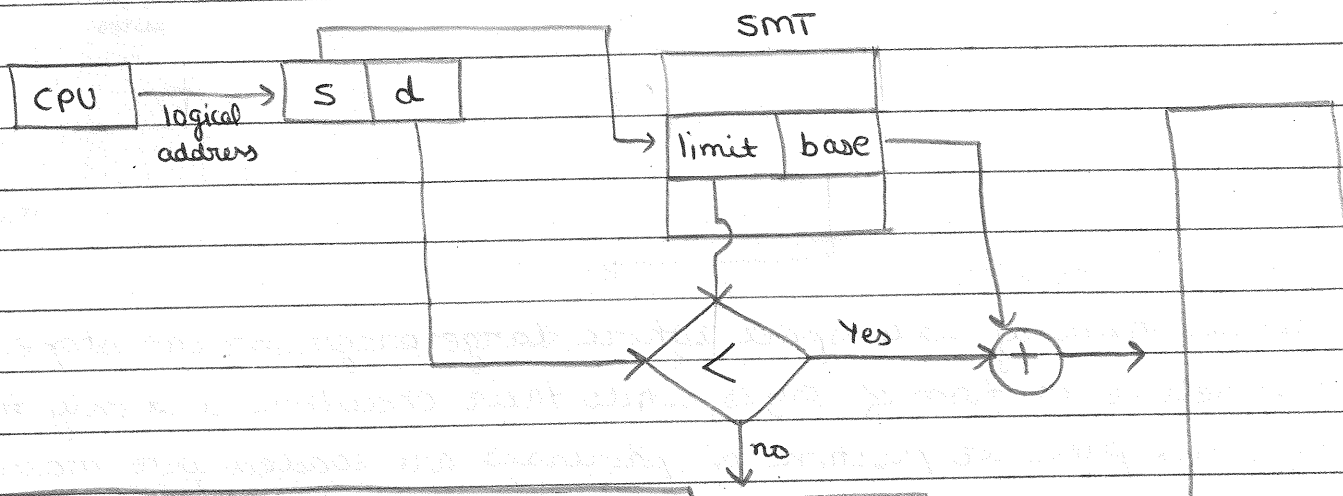
Segmentation.

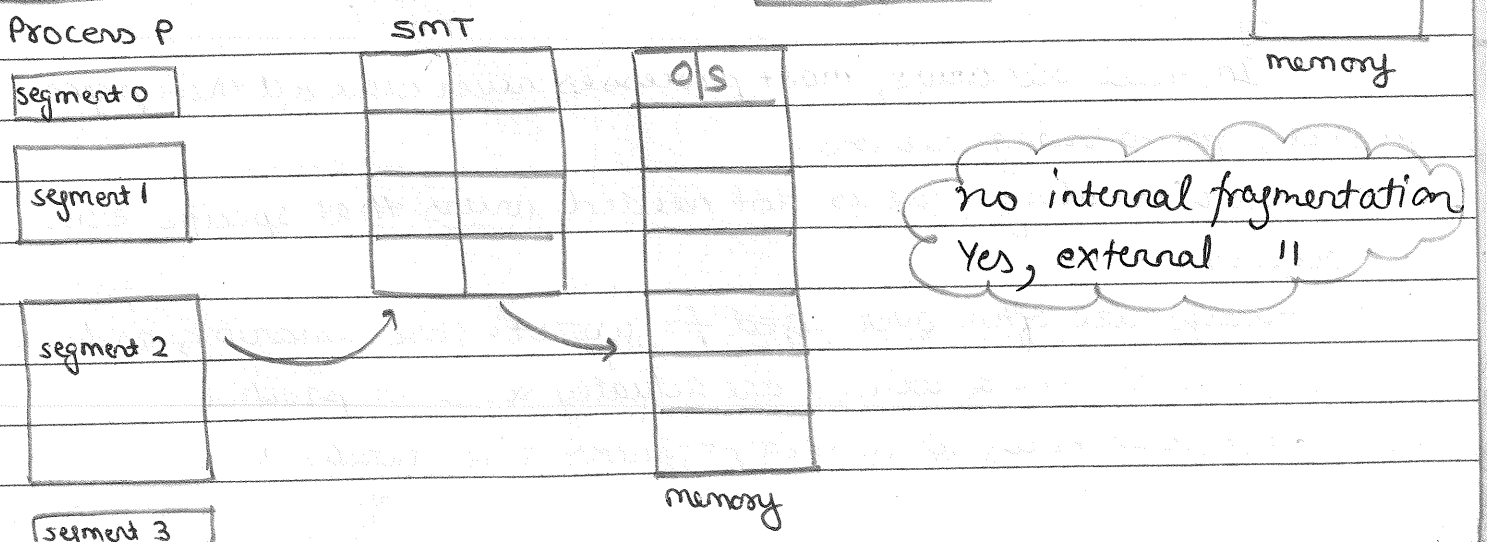
- supports user view of memory
- allows breaking of virtual address space of single process into segments that may be placed in non-contiguous areas of physical memory.

Segmentation is a memory management technique in which each process is divided into several segments of different sizes. Each segment is actually a different logical address space of program.

When a process is to be executed, its corresponding segments are loaded into non-contiguous memory though every segment is loaded into a contiguous block of available memory. The operating system maintains a SMT (Segment Map Table) for every process.

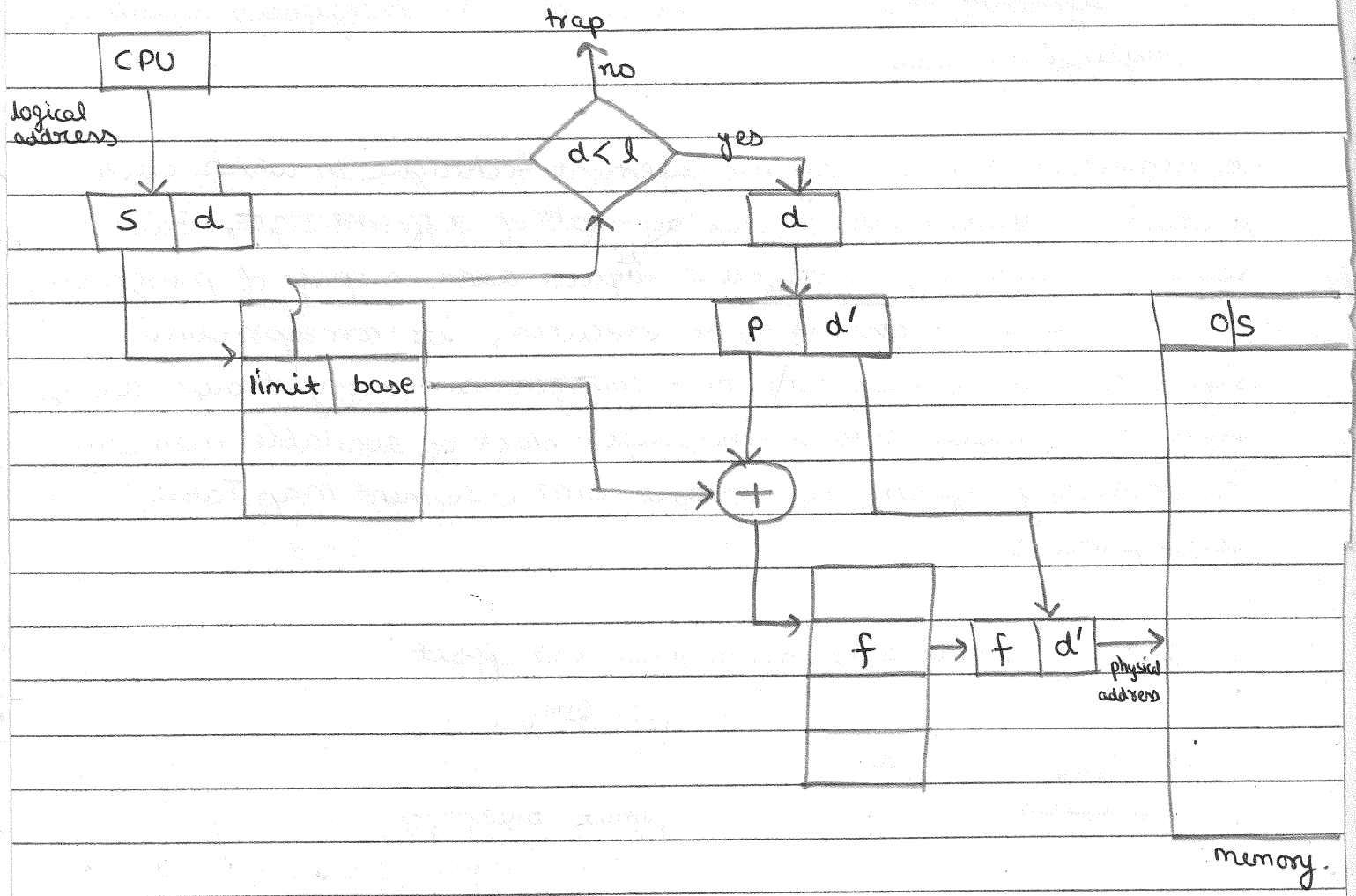
L → s, d where s is segment no and offset





Segmentation with Paging

Both segmentation and paging have its advantages and disadvantages. So, it is better to combine both.



Virtual memory

Virtual memory is a space where large programs can store themselves in form of pages, while their execution and only the required pages or portions of processes are loaded into main memory.

In real scenarios, most processes never need all their pages at once, for following reasons:

(i) Error handling code is not needed unless that specific error occurs

(ii) Arrays are often over-sized for worst-case scenarios, and only a small fraction of arrays are actually used in practice.

(iii) Certain features of certain programs are rarely used.

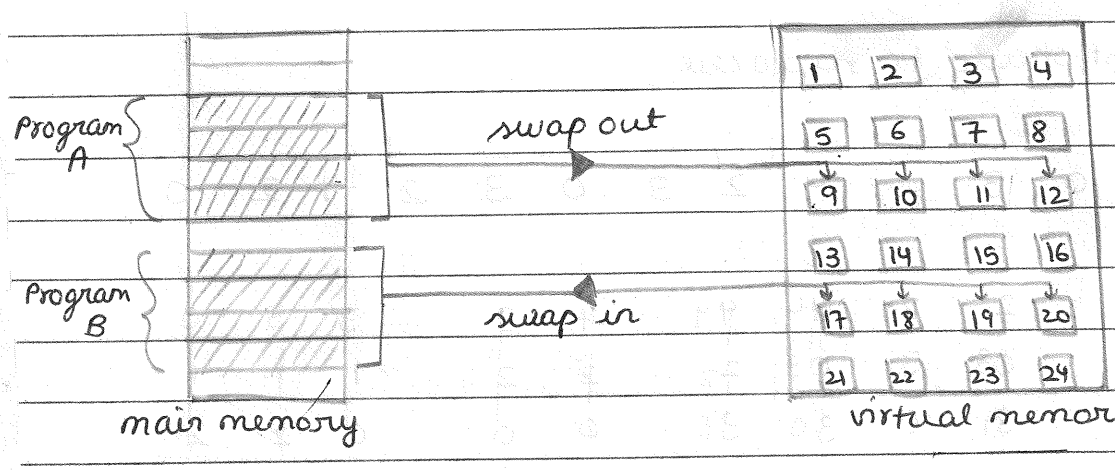
amount of virtual memory depends on secondary storage.

Benefits of Virtual Memory (usually implemented using demand paging)

- Large programs can be written, as virtual space available is huge compared to physical memory
- Less I/O required, leads to faster and easy swapping of processes.
- more physical memory available, as programs are stored on virtual memory, so they occupy very less space on actual physical memory.

Demand Paging (combination of swapping & paging)

The basic idea behind demand paging is that when a process is swapped in, its pages are not swapped in all at once. Rather they are swapped in only when the process needs them (on demand). This is termed as lazy swapper, although a pager is more accurate term.



effective access time = (1-p)ma + p x page fault time

where p is probability of page fault, ma is memory access time.

The pages that are not moved into memory, are marked as invalid in page table. For an invalid entry the rest of the table is empty. In case of pages are loaded in main memory, they are marked as valid along with the information about where to find the swapped out page

There are cases when no pages are loaded into memory initially, pages are loaded when demanded by process by generating page faults. This is called Pure Demand Paging

The major issue with Demand Paging is, after a new page is loaded, the process starts execution from the beginning. Its not a big issue for small programs, but for large programs its ~~performs~~ affects performance only.

Page Replacement

only certain pages of a process are loaded initially into memory. This allows us to get more no. of pages processes into memory at same time. But what happens when a process request for more pages and no free memory is available to bring them in. So, we need to find some pages that are not being used right now, move them to disk to get free frames. This technique is called Page Replacement

Page Replacement Algorithm.

(i) FIFO (first in first out)

Page bought first should be replaced.

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1
7	7	7	2		2	2	4	4	4	0			0	0		
	0	0	0		3	3	3	2	2	2			1	1		
		1	1		1	0	0	0	3	3			3	2		

Page faults = 12 Hits = $\frac{5}{17}$ miss = $\frac{12}{17}$

(ii) Optimal

replace the page which is will not be refered for max duration into future.

best to implement but hard to implement

Dt.:
Pg.:

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1
7	7	7	2		2		2			2			2			
	0	0	0		0		4			0			0			
		1	1		3		3			3			1			

miss = 8/17 hit = 9/17

access time = $ps + (1-p)m$

p = page fault rate
s = page fault handling/service time
m = main memory access time

(iii) LRU (least recently used)
- remove page that was least recently used in past.

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1
7	7	7	2		2		4	4	4	0			1		1	
	0	0	0		0		0	0	3	3			3		0	
		1	1		3		3	2	2	2			2		2	

miss = 11/17 hit = 6/17

Best ⟶ Worst
Optimal > LRU > FIFO

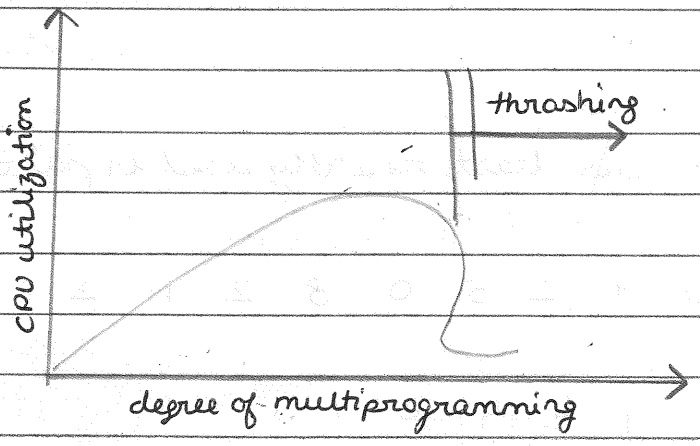
Frame Allocation
(i) max no. of frames that can be alloted to process depends of size of physical memory
(ii) min no. of frames which ..... process depends of Instruction set architecture.

Thrashing

A process that is spending more time paging than executing is said to be thrashing. i.e. process does not have enough frames to hold all pages for its execution, so it is swapping pages in and out very frequently to keep executing. Sometimes, the pages which will be required near future have to be swapped out.

Initially when CPU utilization is low, process scheduling mechanism, to increase level of multiprogramming loads,

multiple processes into memory at same time, allocating a limited amount of frames to each process. As the memory fills up, process starts to spend a lot of time for required pages to be swapped in, again leading to low CPU utilization because most of processes are waiting for pages. Hence the schedular loads more processes to increase CPU utilization, as this continues at a point of time the complete system comes to stop.



To prevent thrashing we must provide processes with as many frames as they really need "right now".

Deadlock avoidance (Banker's Algorithm) (and resource allocation graph).
- is a deadlock avoidance algorithm. It is named so because this algorithm is used in banking system to determine whether a loan can be granted or not

n - processes, m - resource type

Algo:

(i) Request ≤ Need , go to step 2 else error

(ii) Request ≤ Available, go to step 3 else wait

(iii) available = available - request
allocation = allocation + request
need = need - request

(iv) check if this new state is safe ie. a safe sequence exist.

Safety Algo:

(i) work = available , finish = false;

(ii) Find an i such that Finish[i] = false ; need ≤ work;
If no such i, go to step 4.

(iii) work = work + allocation ; finish[i] = True; go to step 2

(iv) If finish[i] = true for all i then system is safe.

q: Resource

A	B	C
10	5	7

resource − allocation = Available

max − allocation = Need

	Allocation			Max			Available			Need		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	3	3	2	7	4	3 (5)
P1	2	0	0	3	2	2	5	4	3 (P3)	1	2	2 (2)
P2	3	0	2	9	0	2	7	4	3 (P1)	6	0	2 (4)
P3	2	1	1	2	2	2	7	4	5 (P4)	0	1	1 (1)
P4	0	0	2	4	3	3	10	4	7 (P2)	4	3	1 (3)
	7	2	5				10	5	7 (P0)			

Check safety algo, safety algo is P_3 P_1 P_4 P_2 P_0 (done with pencil)

Now, P_0 request (A B C) 1 0 2

(i) request ≤ need ie. 1 0 2 ≤ 7 4 3 ✓

(ii) available = available − request = 3 3 2 − 1 0 2

available = 2 3 0

(iii) allocation = allocation + request = 0 1 0 + 1 0 2

allocation = 1 1 2

(iv) need = need − request = 7 4 3 − 1 0 2

need = 6 4 1

Now applying safety algo. If we have safety sequence then we can give P_0 requested resource.

since no safety algo is there, resource to P_0 cannot be allocated.

Dt.:
Pg.:

Comparison of Scheduling Algorithms

CPU can apply different scheduling algorithm to schedule process

FCFS (First come first serve)

advantage
- doesn't include any complex logic
- simple and easy to implement
- starvation doesnot occur.

disadvantage
- no option for pre-emption.
- since no pre-emption, processes in back of queue will have to wait for long time if before process executes for long time.

SJF (shortest job first)

advantage
- shortest process executes first followed by longer process
- throughput increases (more jobs get executed)

disadvantage
- time taken by process must be known by CPU beforehand, which is not possible.
- suffer starvation.

RR (Round Robin)

advantage
- each process is served by CPU for fixed quantum
- starvation doesn't occur

disadvantage
- throughput depends on time quantum. If time quantum is longer than needed, it tends to exhibit same behaviour as FCFS.
- if time quantum is small, no. of CPU switches from one process to another increases, that leads to decrease in CPU efficiency.

Priority based scheduling.

advantage - priority of a process can be selected based on memory requirement, time requirement or user preference.

disadvantage - a second scheduling algorithm is required to schedule processes having same priority.
- a higher priority can execute ahead of an already executing lower priority process. If lower keeps on waiting for higher priority processes, starvation occurs.

Secondary storage structure.

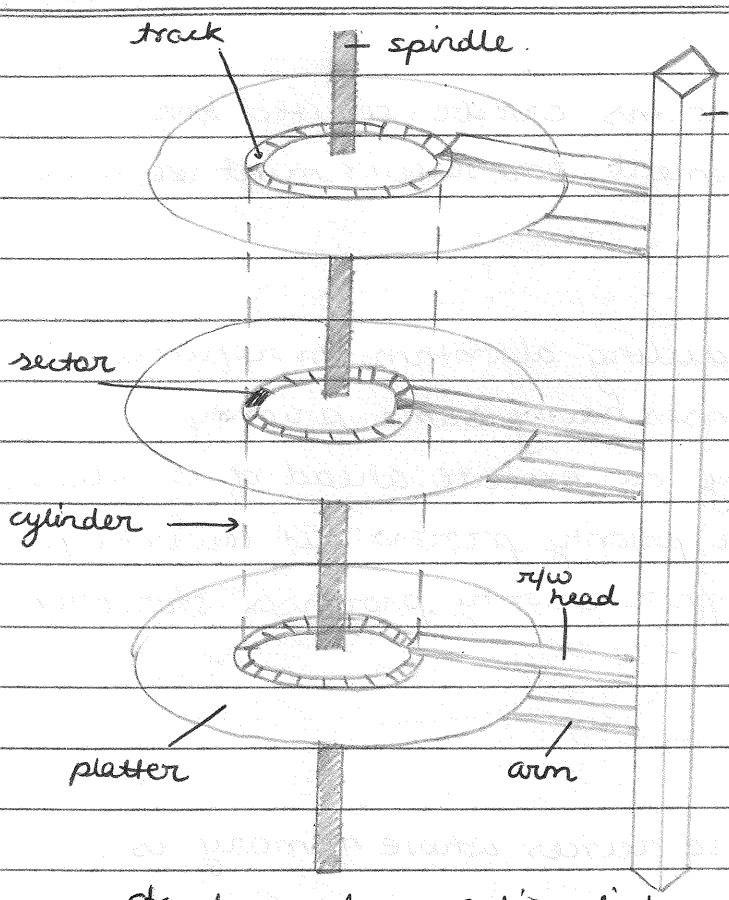
secondary storage devices are those devices whose memory is non-volatile.
- also called auxiliary storage.
- less expensive
- speed is less than main memory
- data which is less frequently accessed is kept in this
- few example are magnetic disks, magnetic tapes, removable thumb drives etc.

Magnetic Disk structure.

In modern computers, most of the secondary storage is in form of magnetic disks.

A magnetic disk contain several platters. Each platter is divided into circular shaped tracks. Each track is further divided into sectors.

A read-write head is used to read data from a sector of magnetic disk.



Structure of magnetic disk

The speed of disk is measured as two parts:
Transfer rate: rate at which data moves from disk to computer
Random access time: sum of seek time and rotational latency.

Seek time is time taken by arm to move to required track.
Rotational latency is time taken by arm to required sector in track.

Even though disk is arranged as sectors and tracks, the data is logically arrayed and addressed as an array of blocks of fixed size. The size of block can be 512 or 1024 bytes. Each logical block is mapped with sector on disk.

Disk Scheduling Algorithms

On a typical multiprogramming system, there will be multiple disk access request at any point of time. So these requests must be scheduled to achieve good efficiency.

Dt.: Pg.: Delta

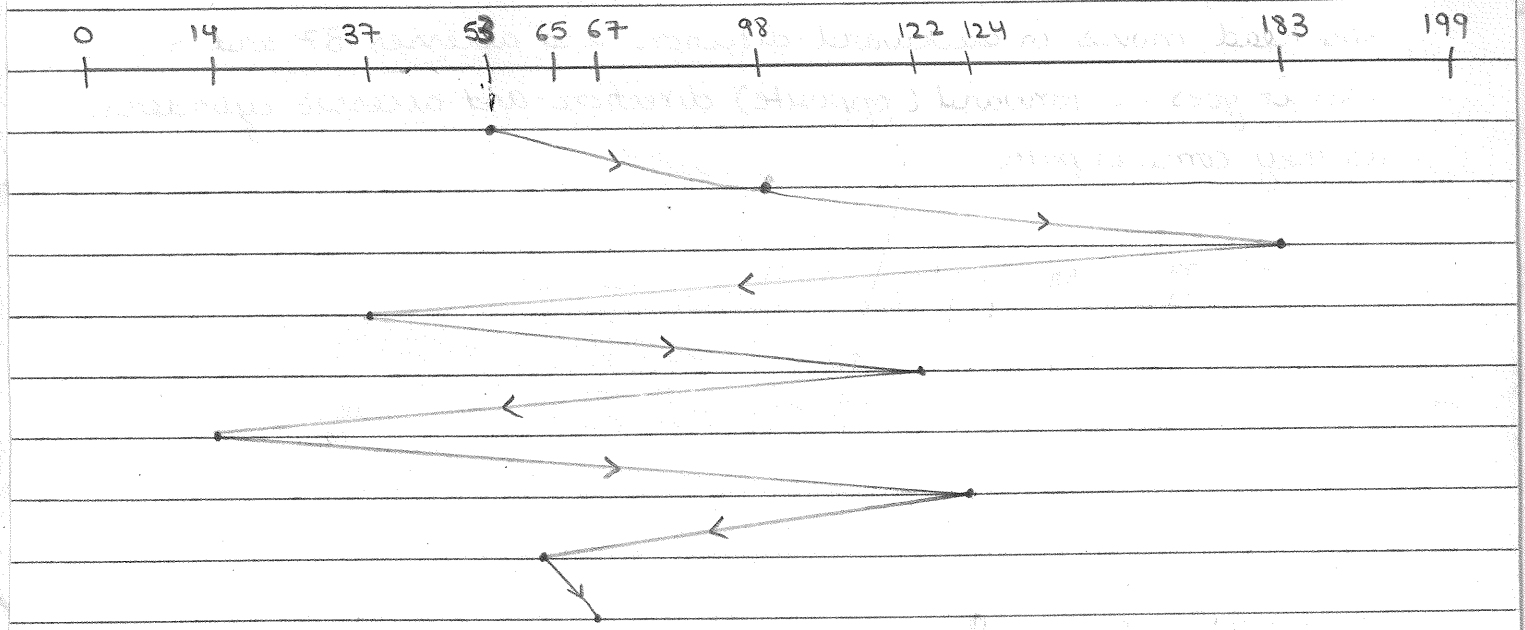
1) First come First serve (FCFS)

Let queue has following requests with cylinder number as follows:

98, 183, 37, 122, 14, 124, 65, 67

Assume head is initially at cylinder 53

The head moves in given order in queue. 56 → 98 → 183 .... → 67

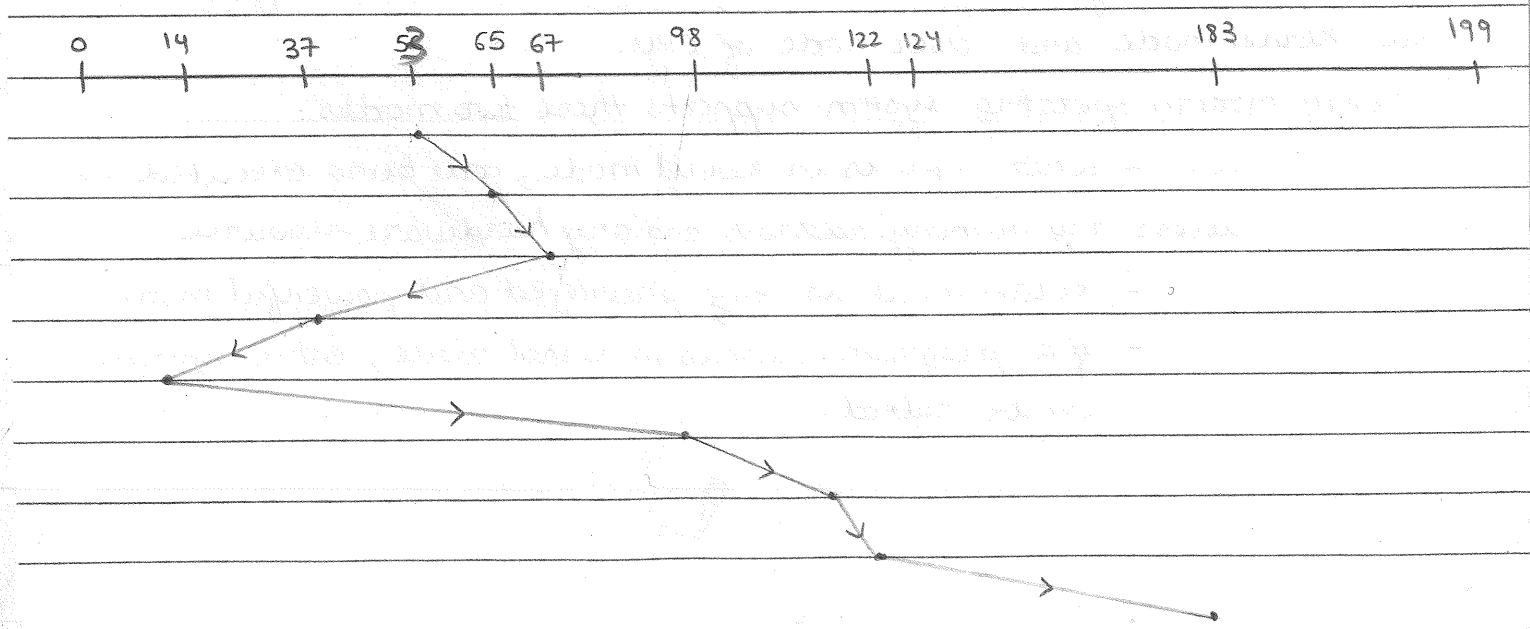


(ii) Shortest Seek Time First (SSTF)

Here where the position which is closest to current head position is chosen first. (question remains same)

So, order will be

53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

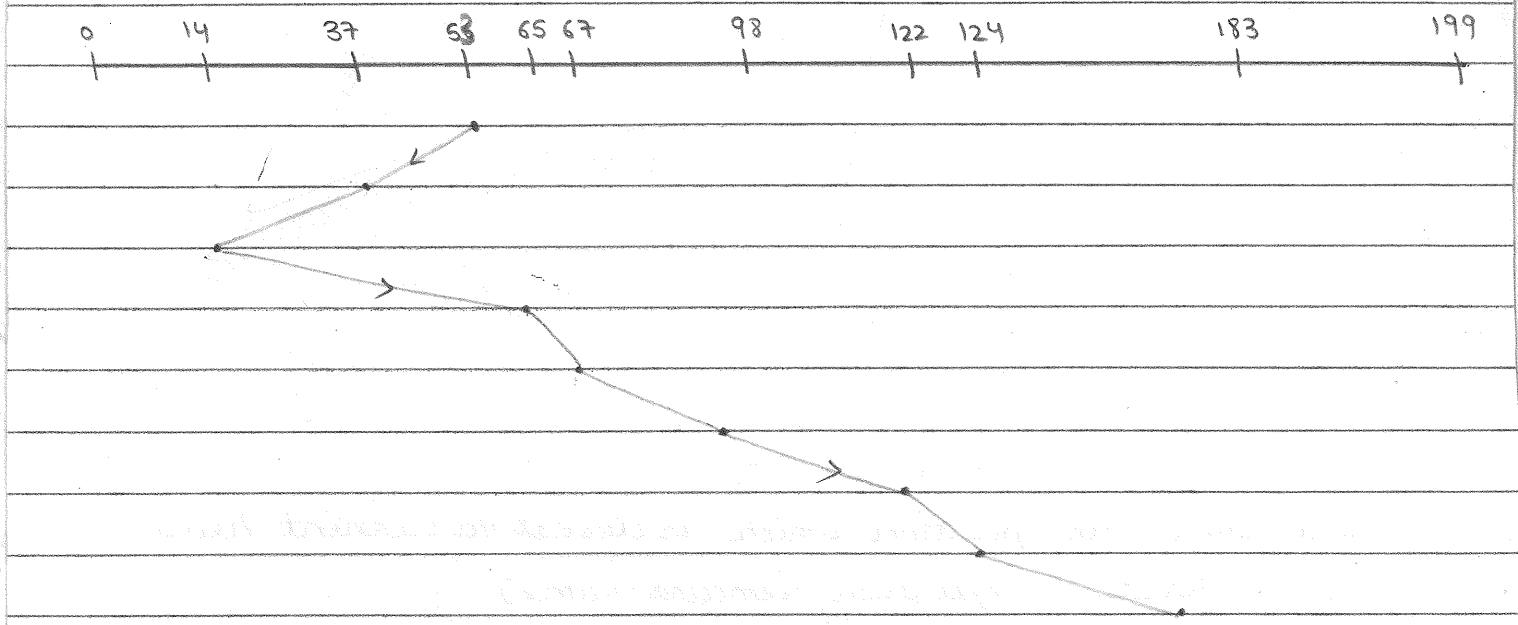


(iii) SCAN algorithm:

This algorithm is also called elevator algorithm because of it's behaviour. Here, first head moves in direction (say backward) and covers all requests in path. Then it moves in ~~remaining~~ opposite direction and covers remaining requests in path.

(question remains same)

The head moves in backward direction and accesses 37 and 14. Then it goes in forward (opposite) direction and accesses cylinders as they come in path.



System Calls

To understand system calls, first we need to understand difference b/w kernel mode and user mode of CPU.

Every modern operating system supports these two modes:

Kernel mode: - when CPU is in kernel mode, code being executed can access any memory address and any hardware resource
- kernel mode is very privileged and powerful mode.
- if a program crashes in kernel mode, entire system will be halted.

bit = 0

Dt.:
Pg.:

User mode – when CPU is in user mode, program don't have direct access to any memory address and hardware resource

bit = 1 – in user mode, if any program crashes, only that particular program is halted. (ie. System will be safe)

– Hence, most program in an OS run in user mode.

System call

When a program in user mode requires access to RAM or hardware resource, it must ask kernel to provide access to that resource. This is done via system call.

When a program ~~is~~ makes a system call, mode is switched from user mode to kernel mode. This is called context switch.

Then the kernel provides resource which program requested. Then another context switch takes place and mode is switched again from kernel mode to user mode.

Some system calls are

(i) fork() – is used to create processes (exact copy)

The changes of values in parent process doesn't affect child and vice versa.

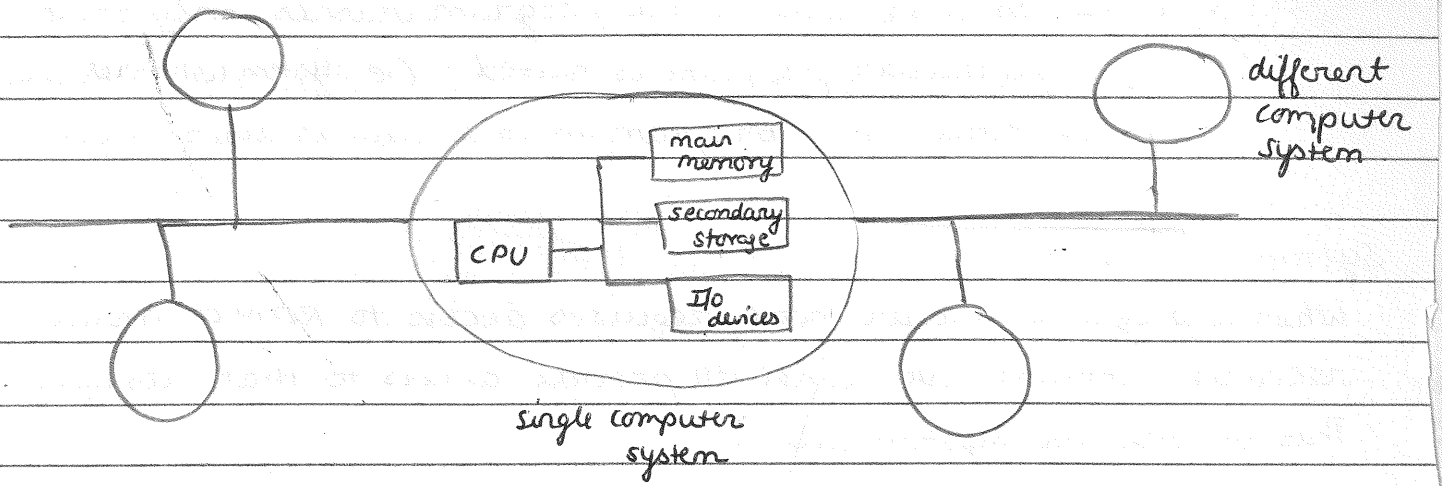
(ii) Exec() – is also used to create processes. In fork() new process is created and child process and parent process both exist. But in Exec() new process is created but destroys parent process and replaces it with new child process.

Dt.:
Pg.:

Distributed computer system:

isolated computers distributed over a network. All are identical.

Nature of work of all computers are identical that can be accessed



We have many computers connected over a network and I have number of jobs to distribute over environment. So which job should be allocated to which computer.

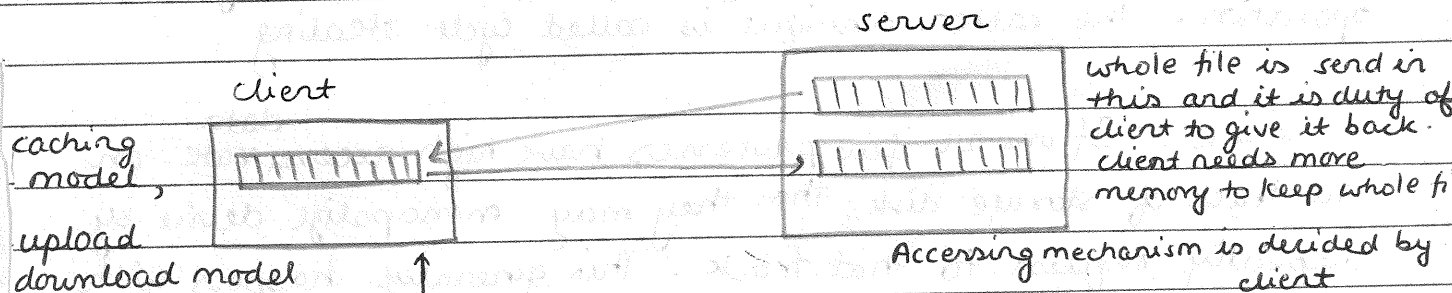
Work station model	Processor pool model
Let us assume LAN model, many work station connected over some LAN. Every user has its computer, CPU, memory, disk etc. ie every user is connected to LAN. User also known as workstation.	You have multiprocessor in one server. User are given high end terminal. User doesnot have any CPU. Processing should be done by server, user just have terminal.
	In short, no user has CPU. He has terminal only and are connected to server. Server has CPU.
In short, every user has CPU and are connected by LAN.	

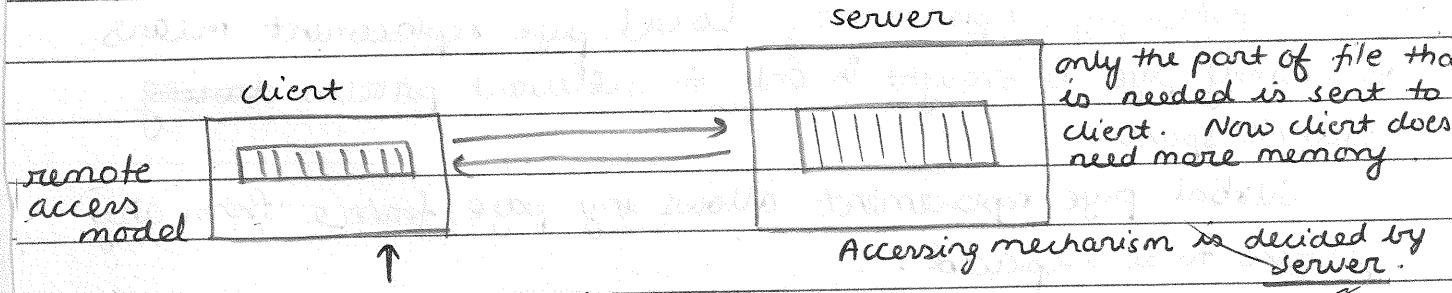
Dt.:
Pg.:

CPU can be given in distributed computing in two ways.

Migratory	Non-migratory.
We can shift from one processor to another.	When you create a process, you then only decide which processor it should execute and it will be finished without preemption ie. cannot be shifted to another processor until it is finished.

When client (not powerful machines) want some file it request server (powerful machine) it is duty of file server to provide it with data.





The child process completes execution, but parent keeps executing, then child process is called Zombie.

Page No.
Date

Some important points.

TLB (Translation Lookaside buffer)
It is a cached system, base addresses of last few referenced pages is maintained in registers called TLB that aids in faster lookup. TLB contains those page entries that have been most recently used.

Local and Global page replacement
Local page replacement means an incoming page is brought in only to relevant process address space.
Global page replacement allows any page frame from any process to be replaced.

Solution to starvation is aging.

UNIX → multiuser, timesharing, multitasking O.S.

Dispatch latency → time taken by dispatcher to stop one process and start another.

Allocation method of disk space → contiguous allocation (Internal / External fragmentation), linked allocation (Internal fragmentation), indexed allocation (no fragmentation)

Bankers Algo is used to avoid deadlock.

Command Interpreter is known as "shell".

Paging is implemented in hardware.

Page No.
Date

Kernel → OS calls.
shell → command is needed for working
↳ program which facilates interaction b/w user and OS.

Belady's Anomaly
proves that if it is possible to have more page faults when increasing no. of page frames in FIFO page replacement algorithm.

eg: 3 2 1 0 3 2 4 3 2 1 0 4

3 frames

	3	2	1	0	3	2	4	3	2	1	0	4
	3	3	3	0	0	0	4			4	4	
		2	2	2	3	3	3			1	1	
Page fault = 9			1	1	1	2	2			2	0	

4 frames

	3	2	1	0	3	2	4	3	2	1	0	4
	3	3	3	3			4	4	4	4	0	0
		2	2	2			2	3	3	3	3	4
			1	1			1	1	2	2	2	2
				0			0	0	0	1	1	1

Page fault = 10

Here, increasing frames increased page fault.

Real core of OS is kernel.
Second important program of OS is User Interface (UI) program.
Old O.S used to have only command line user interface (CLU) called **shell**.

Page No.
Date

Booting of O.S. (ROM)

Once you power-on computer, process of loading O.S and application is called booting.

The basic BIOS program initialises various I/O controllers and I/O devices. Finally, it identifies boot device stored in CMOS memory. Boot devices are floppy disk, harddisk, CD, flash stick and network.

Booting helps to get ROM instructions loaded into main memory.

Cold Booting: When computer is started after having been switched OFF.

warm Booting: when O.S alone is restarted (without being switched OFF) after a system crash or freeze.

Both type of booting clear out the bugs, bombs, memory conflicts of O.S.

Kernel

interface b/w user applications & hardware. When OS is loaded into main memory, kernel loads first and remains in memory until O.S is shut down again.

Types of kernel.

(1) Monolithic kernel: provides rich and powerful abstraction of underlying hardware. O.S is in kernel space (entire OS).
- fast execution, hard to extend
- if service crash, whole system crashes.
- eg: Linux, BSDs.
- user and kernel services are in same address space.

(II) Micro kernel: provide small set of simple hardware abstraction and use applications called servers to provide more functionality. (reduced size of kernel)
- minimum amount of mechanism needed to implement O.S.
- kernel and user services are implemented in different address space.
- slow execution, easily extendible.
- if service crashes, it doesn't effect on working on microkernel.

(III) Hybrid kernel: attempts to combine aspects and benefits of micro and monolithic kernel.

(IV) Exokernel: provides minimal abstraction allowing low level hardware accers. It is designed to seperate resource protection from maragment to faciliate application specific customization.

(V) Nano kernel: is small kernel, code being executed in hardware privileged mode is quite small. It supports nanosecond clock resolution.

Device Drivers.
- software modules that can be plugged into OS to handle particular device. An OS takes help from device drivers to handle all I/O devices.

Page No.
Date

Buffering
↳ technique of storing data in memory area called buffers while data is being transferred b/w two devices or b/w a device and an application program
(i) manages speed mismatch b/w sender & receiver
(ii) maintains balance b/w devices.
(iii) supports copy semantics for an I/O application.

Spooling
process of sending data to spool (buffers or temporary storage area) of computer's memory. OS maintains spooling of data by adding an address location with every element in memory. This data may contain files or processes. It is useful because different devices access data at different rates.
The most common type of spooling is print spooling.

The name of all files created on disk are stored in its directory.
Directory structure is organisation of files into hierarchy of folders.

Authentication: Create a system which ensures user who is running a particular program is authentic.
eg: username/password, user card/key, fingerprint

Authorisation: process of giving someone permission to do or have something. Defines system which users are allowed to system and what privileges of use.